FinanceTracker

Пошаговый план создания настольного приложения для учёта личных финансов

Компонент	Выбор	Назначение
Язык	C#	Основная логика приложения
Платформа	.NET 8 LTS	Стабильная среда выполнения
Интерфейс	Avalonia UI	Нативный интерфейс Linux и Windows
База данных	SQLite	Локальное хранение без отдельного сервера
Архитектура	MVVM	Разделение интерфейса, логики и данных
Тесты	xUnit	Проверка расчётов и работы базы

Оптимальный стек: C# + .NET 8 + Avalonia UI + SQLite. Приложение работает без браузера, хранит данные на компьютере и устанавливается в меню GNOME как обычная программа.

Windows Forms не выбран, потому что официально работает только на Windows. Avalonia сохраняет C# и XAML, но поддерживает Fedora Linux и Windows.

1. Определить минимальную версию (MVP)

- Добавление доходов и расходов.
- Редактирование и удаление операций с подтверждением.
- История операций и фильтр по месяцу.
- Расчёт доходов, расходов и текущего баланса.
- Локальное сохранение после закрытия программы.
- Запуск из меню приложений без браузера.

2. Подготовить инструменты

- .NET 8 SDK для сборки проекта.
- Avalonia Templates для создания интерфейса.
- Visual Studio Code или JetBrains Rider.
- Git для истории изменений и публикации на GitHub.

3. Создать структуру решения

Проект разделяется на интерфейс, бизнес-логику, базу данных и тесты.

```
FinanceTracker/
■■■ src/
■ ■■■ FinanceTracker.App/ — ■■■■ ■ ViewModel
■ ■■■ FinanceTracker.Core/ — ■■■■■■ ■ ■■■■■■■
■ ■■■ FinanceTracker.Data/ — SQLite ■ ■■■■■■■■■■■
■■■ tests/
■ ■■■ FinanceTracker.Tests/ — ■■■■■■■■■■■■■■ ■■■■■
■■■ FinanceTracker.sln
■■■ README.md
■■■ .gitignore
```

4. Создать модель операции

Поля модели: Id, Type, AmountMinor, Category, Description и Date. Type принимает только Income или Expense.

- Сумма хранится в целых копейках: 150,25 грн → 15025.
- Это исключает ошибки округления денежных значений.
- Дата сохраняется в формате ISO 8601: yyyy-MM-dd.

5. Добавить правила проверки

- Сумма должна быть больше нуля.
- Тип и категория обязательны.
- Описание — не более 500 символов.
- Редактировать или удалять можно только существующую запись.

6. Создать SQLite-базу

Файл базы хранится в ~/.local/share/FinanceTracker/finance.db. Он не должен попадать в GitHub.

Таблица	Содержимое
Transactions	Доходы и расходы
Categories	Стандартные и пользовательские категории
Budgets	Лимит расходов по годам и месяцам
Settings	Валюта и пользовательские параметры

7. Реализовать репозиторий

- GetAllAsync — получить операции.
- AddAsync — добавить операцию.
- UpdateAsync — изменить операцию.
- DeleteAsync — удалить операцию.
- GetCategoriesAsync и AddCategoryAsync — работа с категориями.
- GetMonthlyBudgetAsync и SetMonthlyBudgetAsync — работа с бюджетом.
- Использовать параметры SQL, не склеивать запросы строками.

8. Собрать главное окно

- Боковое меню: Главная, Операции, Статистика, Бюджет, Категории, Настройки.
- Карточки: доходы, расходы, баланс и бюджет.
- Последние операции и форма быстрого добавления.
- Тёмная тема с зелёным цветом доходов, красным — расходов и синим — баланса.

9. Реализовать раздел «Операции»

- Полная таблица операций.
- Поиск по категории и описанию.
- Фильтры по месяцу и типу.
- Редактирование выбранной записи.
- Удаление только после подтверждения.
- Сортировка от новых операций к старым.

10. Реализовать статистику

- Суммы доходов и расходов за выбранный месяц.
- Распределение расходов по категориям.
- Процент каждой категории.
- Сравнение фактических расходов с бюджетом.
- Для первой версии достаточно ProgressBar; диаграммы можно добавить позже.

11. Реализовать бюджет

Пользователь выбирает месяц и устанавливает лимит. Приложение показывает потраченную сумму и процент использования. При превышении лимита индикатор становится красным.

12. Реализовать категории

- Расходы: продукты, транспорт, покупки, развлечения, здоровье, коммунальные услуги, рестораны, образование, другое.
- Доходы: зарплата, подработка, подарок, возврат, другое.
- Пользователь может добавлять собственные категории.

13. Добавить настройки

- Выбор обозначения валюты.
- Просмотр пути к базе данных.
- Открытие папки данных.
- Экспорт истории в CSV.
- Создание и восстановление резервной копии.

14. Написать автоматические тесты

- Расчёт доходов, расходов и баланса.
- Округление денежных значений.
- Проверка некорректной суммы и категории.
- Добавление, изменение и удаление записи SQLite.
- Сохранение бюджета и загрузка категорий.

15. Провести ручное тестирование

- Запуск с пустой базой.
- Все CRUD-операции.
- Сохранение после перезапуска.
- Фильтры и поиск.
- Отрицательный баланс и превышение бюджета.
- Запуск установленной версии из меню GNOME.

16. Собрать самостоятельное приложение

Выполнить self-contained публикацию для Linux x64, чтобы пользователю не требовалась отдельная установка .NET.

```
dotnet publish src/FinanceTracker.App -c Release -r linux-x64 --self-contained true
```

17. Установить приложение на ПК

- Программа: ~/.local/lib/FinanceTracker/
- Данные: ~/.local/share/FinanceTracker/finance.db
- Ярлык: ~/.local/share/applications/financetracker.desktop
- Иконка: ~/.local/share/icons/hicolor/scalable/apps/financetracker.svg

18. Оформить GitHub

- README с описанием, скриншотами и инструкцией запуска.
- Описание архитектуры и используемых технологий.
- Список реализованных функций и план развития.
- Не публиковать bin, obj, finance.db и реальные финансовые данные.

Порядок реализации

Этап	Результат
1. Core	Модель, деньги, расчёты и валидация
2. Data	SQLite, CRUD, категории и бюджет
3. App	Главное окно и навигация
4. Features	Операции, фильтры, статистика и экспорт
5. Quality	Автоматические и ручные тесты
6. Delivery	Self-contained сборка, ярлык и GitHub

Итог: C# + .NET 8 + Avalonia UI + SQLite — оптимальная основа для настоящего устанавливаемого FinanceTracker на Fedora Linux и Windows.